Internship Task 2: Camtasia-style zoom effect for recorded videos

Objective

Extend the recorder from the test tasks with a **Camtasia-style zoom effect**: while recording, a zoom can be triggered around the current mouse position (F5 or programmatically); after recording, the video is post-processed so the zoom is rendered smoothly. The post-processing must run inside the Docker stack from **Internship Task 1**.

Architecture

1. **Capture** (Playwright, in `qscript.py`): record zoom events `{t, x, y, type}` during recording and write them to a sidecar file.
2. **Post-process** (`apply_zoom.py`, ffmpeg): read the sidecar and render the video with a smooth zoom-in → hold → zoom-out around each point.
3. **Integration** (Docker): the zoom step runs in the service (worker/endpoint) so an uploaded video can be processed automatically; the result is stored in PostgreSQL.

Constraints

- Python 3.9+
- Playwright (sync API) for capture; ffmpeg (`zoompan`) for post-processing.
- Reuse `qscript.py` / `capture_video.py` from the test tasks and the Dockerized service from Internship Task 1.
- Zoom events are stored as a JSON sidecar next to the video (and/or in PostgreSQL).
- Mouse → video coordinate mapping must be correct.

Deliverables

1. Updated `qscript.py` — zoom-event capture + F5 hotkey + sidecar writing.
2. `apply_zoom.py` — ffmpeg post-processing.
3. Integration into the Docker service (processing step + ffmpeg in the image).
4. Tests (pytest) for the zoom logic.
5. Updated `Dockerfile` / `docker-compose.yml` / `README`.

1. Zoom capture (during recording)

- Add an **F5** hotkey that toggles the zoom: the first press marks the **start of the zoom-in** at the current mouse position, the second press marks the **start of the zoom-out**.
- Expose a programmatic hook `window.__zoomAt(x, y)` so the automation scenario can trigger the same toggle (automation does not press F5).
- Show a brief on-screen indicator (a colored ring) so the operator sees the toggle state.
- Each event is `{"t": <seconds since video start>, "x": <px>, "y": <px>, "type": "in" | "out"}`.
- When at least one event exists, write a sidecar `<video>.zoom.json`:

```
{
  "viewport": {"width": 1280, "height": 720},
  "fps": 25,
  "zoom_factor": 2.0,
  "ramp_sec": 0.5,
  "events": [
    {"t": 3.0, "x": 640, "y": 360, "type": "in"},
    {"t": 8.0, "x": 640, "y": 360, "type": "out"}
  ]
}
```

2. Zoom post-processing (ffmpeg)

`apply_zoom.py` reads the sidecar and applies the `zoompan` filter:

- Map mouse coords to video pixels (1:1 because `record_video_size` == viewport).
- Pair each `in` with the next `out`: zoom-in (ramp) → hold (user-controlled) → zoom-out (ramp), clamping the crop to the frame.
- Support multiple, non-overlapping in/out pairs.
- If an `in` has no following `out` (recording ended while zoomed), hold the zoom to the end of the video.
- Output a new `.webm` (e.g. `libvpx-vp9`), preserving the input resolution.
- Provide a `--dry-run` flag to preview the ffmpeg command, and a `--ffmpeg <path>` option to point at the ffmpeg binary when it is not on `PATH`.

```
python apply_zoom.py --video video/ACME_GmbH.webm
python apply_zoom.py --video video/ACME_GmbH.webm --dry-run
```

3. Integration with the Docker service (Task 1)

- Install ffmpeg in the relevant image (the `web`/`worker` container).
- Add a processing step so an uploaded video can be zoomed automatically — a background job, a `POST /jobs/<id>/process` endpoint, or a worker that picks up pending videos.
- Store the processed video URL in PostgreSQL (e.g. a `processed_video_url` column or a `jobs` table with status).

4. Tests (pytest)

- Unit-test the zoom-filter generation and coordinate math (no ffmpeg required): ramp frame calculation, clamping, multiple in/out pairs, and an open-ended zoom (an `in` with no `out` holds to the end).
- Test sidecar parsing, old sidecars without `type`, and the "no events → copy unchanged" path.
- (Integration) verify the Docker stack installs ffmpeg and the processing step works.

Acceptance checklist

- [ ] The first F5 marks the start of the zoom-in; the second F5 marks the start of the zoom-out.
- [ ] `window.__zoomAt(x, y)` toggles in/out from the scenario.
- [ ] `<video>.zoom.json` is written only when there is at least one event.
- [ ] `python apply_zoom.py --video ...` produces a zoomed `.webm`.
- [ ] `apply_zoom.py --dry-run` prints the ffmpeg command without encoding.
- [ ] An open-ended zoom (an `in` without a following `out`) holds to the end of the video.
- [ ] The zoom centers on the exact mouse point and stays within frame bounds.
- [ ] The effect is smooth (ease-in, hold, ease-out) and supports multiple in/out pairs.
- [ ] The post-processing runs inside the Docker stack and stores the result in PostgreSQL.
- [ ] Tests pass locally (and in CI if configured).

Hints / pitfalls

- `record_video_dir` records a fixed viewport; zoom is a **post effect**, not a live zoom.
- Keep `record_video_size` == viewport size so mouse pixels map 1:1 to video pixels.
- Event timestamps are relative to the video start; use the context-creation moment as `t=0`, or inject a visible marker at `t=0` to calibrate.
- In `zoompan`, the crop is `iw/zoom × ih/zoom`; center it with `x = clamp(px - iw/(2*zoom), 0, iw - iw/zoom)` (same for `y`).
- Linear ramps look robotic — add easing to the ramp in/out. (The hold duration is the time between the two F5 presses.)

- Do not write the sidecar unless there are events, so normal recordings are unaffected.
- ffmpeg must be present in the processing container image (add it in the Dockerfile), or point `apply_zoom.py` at it with `--ffmpeg <path>`.